
Problem Statements

1. OOP Fundamentals (Classes, Inheritance, Encapsulation, Constructors, Methods)

Problem Description:

Create a class `Employee` with private attributes `id`, `name`, and `salary`. Implement constructors, getter/setter methods, and a method `displayDetails()`. Derive a subclass `Manager` that adds an attribute `department`. Override `displayDetails()` to include department information.

Input Format:

- First line: Employee ID (integer)
- Second line: Employee Name (string)
- Third line: Salary (double)
- Fourth line: Department (string)

Output Format:

- Display employee details using `displayDetails()` method.

Constraints:

- Salary must be positive.
- Name cannot be empty.

Sample Input:

101

Alice

50000

HR

Sample Output:

ID: 101

Name: Alice

Salary: 50000.0

Department: HR

2. Collections & Arrays (ArrayList, HashMap, arrays)

Problem Description:

Write a program to store student names and marks using a `HashMap<String, Integer>`. Read n students, then print the student with the highest marks. Also, store all names in an `ArrayList` and print them in sorted order.

Input Format:

- First line: Number of students n
- Next n lines: Name and Marks separated by space

Output Format:

- Highest scoring student name and marks
- Sorted list of student names

Constraints:

- Marks must be between 0 and 100.

Sample Input:

3

John 85

Alice 92

Bob 78

Sample Output:

Topper: Alice with 92 marks

Sorted Names: [Alice, Bob, John]

3. String Parsing & Type Conversion

Problem Description:

Write a program that reads a string containing comma-separated integers. Parse the string, convert each to integer, and calculate their sum.

Input Format:

- A single line containing comma-separated integers.

Output Format:

- Print the sum of integers.

Constraints:

- Input string will contain only digits and commas.

Sample Input:

10,20,30,40

Sample Output:

Sum = 100

4. Control Flow (Loops, Conditionals, Flags)**Problem Description:**

Write a program to read an integer n and print whether it is prime or not using loops and conditionals.

Input Format:

- Single integer n

Output Format:

- Print "Prime" or "Not Prime"

Constraints:

- $2 \leq n \leq 1000$

Sample Input:

17

Sample Output:

Prime

5. Validation & Constraints Checking**Problem Description:**

Write a program to validate a password. A valid password must:

- Be at least 8 characters long
- Contain at least one uppercase letter

- Contain at least one digit

Print "Valid Password" or "Invalid Password".

Input Format:

- Single string (password)

Output Format:

- Validation result

Sample Input:

Pass1234

Sample Output:

Valid Password

6. Algorithmic Logic (Sorting, Palindrome, Filtering, Calculations)

Problem Description:

Write a program to read n integers into an array.

- Sort the array in ascending order.
- Print all palindrome numbers from the array.

Input Format:

- First line: Number of integers n
- Next line: n integers separated by space

Output Format:

- Sorted array
- Palindrome numbers

Sample Input:

5

121 34 44 78 11

Sample Output:

Sorted Array: [11, 34, 44, 78, 121]

Palindromes: [121, 44, 11]

7. Input/Output Handling (Scanner, System.out.println)

Problem Description:

Write a program that reads a user's name and age using Scanner. Print a greeting message.

Input Format:

- First line: Name (string)
- Second line: Age (integer)

Output Format:

- Greeting message

Sample Input:

Alice

23

Sample Output:

Hello Alice, you are 23 years old!

Integrated Problem Statement

Problem Description

You are tasked with building a **Student Management System** in Java. The system should allow users to add students, validate their details, store them in collections, perform algorithmic operations, and display results. The program must demonstrate **OOP fundamentals, collections, string parsing, control flow, validation, algorithmic logic, and I/O handling**.

Requirements:

1. OOP Fundamentals

- Create a class Student with attributes: id, name, age, and marks.
- Use **constructors, encapsulation (private fields + getters/setters)**, and a method displayDetails().
- Create a subclass GraduateStudent that adds specialization and overrides displayDetails().

2. Collections & Arrays

- Store students in an `ArrayList<Student>`.
- Maintain a `HashMap<Integer, Student>` for quick lookup by ID.
- Use arrays for sorting marks.

3. String Parsing & Type Conversion

- Accept student details in a single comma-separated string (e.g., "101,Alice,21,85").
- Parse and convert values to appropriate types.

4. Control Flow

- Use loops and conditionals to process multiple students.
- Implement a flag to stop input when the user types "exit".

5. Validation & Constraints Checking

- Age must be ≥ 18 .
- Marks must be between 0 and 100.
- Name cannot be empty.

6. Algorithmic Logic

- Sort students by marks.
- Identify and print all students whose names are palindromes.
- Calculate the average marks of all students.

7. Input/Output Handling

- Use `Scanner` for input.
- Use `System.out.println` for output.

Input Format

- Each line: id,name,age,marks
- Input continues until "exit" is entered.

Output Format

- Display all student details.

- Print sorted list of students by marks.
- Print palindrome names.
- Print average marks.

Constraints

- $1 \leq \text{id} \leq 9999$
- $\text{Age} \geq 18$
- Marks between 0 and 100

Sample Input

101,Alice,21,85

102,Bob,19,92

103,Ana,22,44

exit

Sample Output

Student Details:

ID: 101, Name: Alice, Age: 21, Marks: 85

ID: 102, Name: Bob, Age: 19, Marks: 92

ID: 103, Name: Ana, Age: 22, Marks: 44

Sorted by Marks:

Ana - 44

Alice - 85

Bob - 92

Palindrome Names:

Ana

Problem Statement 2: Shopping Cart System

Problem Description

You are tasked with building a **Shopping Cart System** in Java. The program should allow users to add products, validate their details, store them in collections, perform algorithmic operations, and display results. The solution must demonstrate **OOP fundamentals, collections, string parsing, control flow, validation, algorithmic logic, and I/O handling**.

Requirements:

1. OOP Fundamentals

- Create a class Product with attributes: id, name, price, and quantity.
- Use **constructors, encapsulation (private fields + getters/setters)**, and a method displayDetails().
- Create a subclass DiscountedProduct that adds a discountPercentage attribute and overrides displayDetails() to show discounted price.

2. Collections & Arrays

- Store products in an ArrayList<Product>.
- Maintain a HashMap<Integer, Product> for quick lookup by product ID.
- Use arrays to sort product prices.

3. String Parsing & Type Conversion

- Accept product details in a single comma-separated string (e.g., "201,Laptop,55000,2").
- Parse and convert values to appropriate types.

4. Control Flow

- Use loops and conditionals to process multiple products.
- Implement a flag to stop input when the user types "checkout".

5. Validation & Constraints Checking

- Price must be positive.

- Quantity must be ≥ 1 .
- Name cannot be empty.

6. Algorithmic Logic

- Sort products by price.
- Identify and print all products whose names are palindromes.
- Calculate the total bill amount (considering discounts if applicable).

7. Input/Output Handling

- Use Scanner for input.
- Use System.out.println for output.

Input Format

- Each line: id,name,price,quantity
- Input continues until "checkout" is entered.

Output Format

- Display all product details.
- Print sorted list of products by price.
- Print palindrome product names.
- Print total bill amount.

Constraints

- $1 \leq \text{id} \leq 9999$
- Price > 0
- Quantity ≥ 1

Sample Input

201,Laptop,55000,2

202,Phone,30000,1

203,Madam,1500,3

checkout

Sample Output

Product Details:

ID: 201, Name: Laptop, Price: 55000.0, Quantity: 2

ID: 202, Name: Phone, Price: 30000.0, Quantity: 1

ID: 203, Name: Madam, Price: 1500.0, Quantity: 3

Sorted by Price:

Madam - 1500.0

Phone - 30000.0

Laptop - 55000.0

Palindrome Product Names:

Madam

Total Bill = 116500.0



Problem Statement 3: Library Management System

Problem Description

You are tasked with building a **Library Management System** in Java. The program should allow users to add books, validate their details, store them in collections, perform algorithmic operations, and display results. The solution must demonstrate **OOP fundamentals, collections, string parsing, control flow, validation, algorithmic logic, and I/O handling**.

Requirements:

1. OOP Fundamentals

- Create a class Book with attributes: id, title, author, and price.

- Use **constructors, encapsulation (private fields + getters/setters)**, and a method `displayDetails()`.
- Create a subclass `EBook` that adds an attribute `fileSizeMB` and overrides `displayDetails()`.

2. Collections & Arrays

- Store books in an `ArrayList<Book>`.
- Maintain a `HashMap<Integer, Book>` for quick lookup by book ID.
- Use arrays to sort book prices.

3. String Parsing & Type Conversion

- Accept book details in a single comma-separated string (e.g., "301,Java Basics,John Doe,450").
- Parse and convert values to appropriate types.

4. Control Flow

- Use loops and conditionals to process multiple books.
- Implement a flag to stop input when the user types "done".

5. Validation & Constraints Checking

- Price must be positive.
- Title and author cannot be empty.
- ID must be unique.

6. Algorithmic Logic

- Sort books by price.
- Identify and print all books whose titles are palindromes.
- Calculate the average price of all books.

7. Input/Output Handling

- Use `Scanner` for input.
- Use `System.out.println` for output.

Input Format

- Each line: `id,title,author,price`

- Input continues until "done" is entered.

Output Format

- Display all book details.
 - Print sorted list of books by price.
 - Print palindrome book titles.
 - Print average book price.
-

Constraints

- $1 \leq \text{id} \leq 9999$
 - $\text{Price} > 0$
 - Title and author cannot be empty
-

Sample Input

301,Java Basics,John Doe,450

302,Madam,Jane Smith,300

303,Algorithms,Alan Turing,600

done

Sample Output

Book Details:

ID: 301, Title: Java Basics, Author: John Doe, Price: 450.0

ID: 302, Title: Madam, Author: Jane Smith, Price: 300.0

ID: 303, Title: Algorithms, Author: Alan Turing, Price: 600.0

Sorted by Price:

Madam - 300.0

Java Basics - 450.0

Algorithms - 600.0

Palindrome Titles:

Madam

Average Price = 450.0

Problem Statement 4: Bank Account Management System

Problem Description

You are tasked with building a **Bank Account Management System** in Java. The program should allow users to create accounts, validate their details, store them in collections, perform algorithmic operations, and display results. The solution must demonstrate **OOP fundamentals, collections, string parsing, control flow, validation, algorithmic logic, and I/O handling**.

Requirements:

1. OOP Fundamentals

- Create a class `BankAccount` with attributes: `accountNumber`, `holderName`, `balance`.
- Use **constructors, encapsulation (private fields + getters/setters)**, and a method `displayDetails()`.
- Create a subclass `SavingsAccount` that adds an attribute `interestRate` and overrides `displayDetails()`.

2. Collections & Arrays

- Store accounts in an `ArrayList<BankAccount>`.
- Maintain a `HashMap<Integer, BankAccount>` for quick lookup by account number.
- Use arrays to sort balances.

3. String Parsing & Type Conversion

- Accept account details in a single comma-separated string (e.g., `"1001,John,5000"`).
- Parse and convert values to appropriate types.

4. Control Flow

- Use loops and conditionals to process multiple accounts.
- Implement a flag to stop input when the user types "exit".

5. Validation & Constraints Checking

- Balance must be ≥ 0 .
- Holder name cannot be empty.
- Account number must be unique.

6. Algorithmic Logic

- Sort accounts by balance.
- Identify and print all accounts whose holder names are palindromes.
- Calculate the total balance across all accounts.

7. Input/Output Handling

- Use Scanner for input.
- Use System.out.println for output.

Input Format

- Each line: accountNumber,holderName,balance
- Input continues until "exit" is entered.

Output Format

- Display all account details.
- Print sorted list of accounts by balance.
- Print palindrome holder names.
- Print total balance.

Constraints

- $1 \leq \text{accountNumber} \leq 99999$
 - Balance ≥ 0
 - Holder name cannot be empty
-

Sample Input

1001,John,5000

1002,Ana,3000

1003,Madam,7000

exit

Sample Output

Account Details:

Account Number: 1001, Holder: John, Balance: 5000.0

Account Number: 1002, Holder: Ana, Balance: 3000.0

Account Number: 1003, Holder: Madam, Balance: 7000.0

Sorted by Balance:

Ana - 3000.0

John - 5000.0

Madam - 7000.0

Palindrome Holder Names:

Ana

Madam

Total Balance = 15000.0

This **Bank Account Management System** is another integrated project that forces learners to apply all the Java fundamentals in a financial context.



Problem Statement 5: Hospital Patient Management System

Problem Description

You are tasked with building a **Hospital Patient Management System** in Java. The program should allow users to register patients, validate their details, store them in collections, perform algorithmic operations, and display results. The solution must demonstrate **OOP fundamentals, collections, string parsing, control flow, validation, algorithmic logic, and I/O handling**.

Requirements:

1. OOP Fundamentals

- Create a class Patient with attributes: id, name, age, disease.
- Use **constructors, encapsulation (private fields + getters/setters)**, and a method displayDetails().
- Create a subclass InPatient that adds attributes roomNumber and daysAdmitted, overriding displayDetails().

2. Collections & Arrays

- Store patients in an ArrayList<Patient>.
- Maintain a HashMap<Integer, Patient> for quick lookup by patient ID.
- Use arrays to sort patients by age.

3. String Parsing & Type Conversion

- Accept patient details in a single comma-separated string (e.g., "501,John,45,Flu").
- Parse and convert values to appropriate types.

4. Control Flow

- Use loops and conditionals to process multiple patients.
- Implement a flag to stop input when the user types "stop".

5. Validation & Constraints Checking

- Age must be ≥ 0 .
- Name cannot be empty.
- Disease cannot be empty.
- ID must be unique.

6. Algorithmic Logic

- Sort patients by age.

- Identify and print all patients whose names are palindromes.
- Calculate the average age of all patients.

7. Input/Output Handling

- Use Scanner for input.
- Use System.out.println for output.

Input Format

- Each line: id,name,age,disease
- Input continues until "stop" is entered.

Output Format

- Display all patient details.
- Print sorted list of patients by age.
- Print palindrome patient names.
- Print average age.

Constraints

- $1 \leq \text{id} \leq 9999$
- $\text{Age} \geq 0$
- Name and disease cannot be empty

Sample Input

501,John,45,Flu

502,Ana,30,Cold

503,Madam,50,Fever

stop

Sample Output

Patient Details:

ID: 501, Name: John, Age: 45, Disease: Flu

ID: 502, Name: Ana, Age: 30, Disease: Cold

ID: 503, Name: Madam, Age: 50, Disease: Fever

Sorted by Age:

Ana - 30

John - 45

Madam - 50

Palindrome Names:

Ana

Madam

Average Age = 41.67
